

Agent Skills × Spring Boot

How **Addy Osmani's agent-skills** make an AI coding assistant build a Spring Boot feature the way a **senior developer** would: plan first, small steps, tests every time.

Spring Boot 3.5

Java 21

Maven

Claude Code

45 tests

[View the demo repo ↗](#)

[addyosmani/agent-skills ↗](#)

The idea

An eager intern vs a senior developer

Same AI, same request. The only difference is a set of instructions it follows.

WITHOUT SKILLS

- "Build an orders API"
- Writes all the code at once
- Hopes it works

Big diff. Few tests. Hidden bugs.

WITH AGENT-SKILLS

- "Build an orders API"
- Asks questions, writes a spec, makes a plan
- Builds one small task at a time, with tests

Small reviewed commits. Proven behaviour.

What's in the box

A **skill** is just a text file of instructions, like a recipe card. The AI picks the right one for the job.

25

Skills

Recipe cards. Like your team's runbooks and coding standards.

test-driven-development
• api-and-interface-design
• security-and-hardening

9

Commands

Shortcut buttons. Like a Maven goal that runs a whole phase.

/spec • /plan • /build •
/test • /review • /ship

4

Agents

Expert reviewers. Like asking a specialist colleague.

code-reviewer • test-engineer • security-auditor

The workflow

Six steps, in order

1

THINK

Spec

/agent-skills:spec

A design doc: what to build and what "done" means

2

THINK

Plan

/agent-skills:plan

A to-do list of small tasks

3

DO

Build

/agent-skills:build

Code and tests for one task, then it stops

4

DO

Test

/agent-skills:test

A failing test first, then the code

5

CHECK

Review

/agent-skills:review

A list of problems to fix

6

CHECK

Ship

/agent-skills:ship

A go / no-go checklist

Tip Always type the full name, like `/agent-skills:plan`. Claude Code has its own built-in `/plan` that does something different.

A tiny shop with one new feature: orders

The app started with products only. The orders feature was built entirely through the six steps.

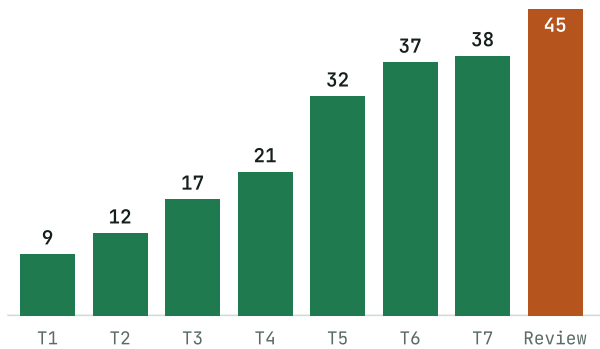
CUSTOMER DOES	APP ANSWERS	WHY
Orders 2 keyboards	201 Created	Order saved, stock goes 25 → 23
Reads the order back	200 OK	Same body as when it was created
Orders a product that doesn't exist	404 Not Found	"Product 99 not found"
Orders 10 monitors, only 8 in stock	409 Conflict	"requested 10, available 8"
Sends broken or empty data	400 Bad Request	Lists each field that is wrong
Something unexpected breaks	500 Error	Logged on the server; no internals shown

What actually happened

Built in small, tested steps

Each `/build` run did one task, ran `mvn test`, committed, and stopped for a human check.

The test suite grew with every step.



Automated tests after each step. Bar height is proportional to the count.

- spec
Asked questions, wrote SPEC.md
- plan
7 tasks, hardest one first
- T1-T2
Stock locking, order tables
- T3-T4
Place an order, read it back
- T5-T7
Clear errors: 400, 404, 409
- review
Found 3 problems in its own work, fixed each
- ship
Checklist of what's needed before real customers
- merge
feature/orders → main

The biggest lesson

Green tests don't always mean working code

During the exercise one annotation was deleted from `OrderService` :

```
@Transactional  
public OrderResponse placeOrder(PlaceOrderRequest request) { ... }
```

UNIT TESTS WITH MOCKS

38 / 38

All still passed. The tests used fake ("mock") databases, so they could not see that nothing was being saved.

THE REAL RUNNING APP

20 / 20

buyers got the **last** monitor. Stock never went down, because without a transaction the changes were never written.

The review caught it. The fix was **one integration test that uses the real database**. Now deleting that line makes the build fail, as it should.

Two people, one monitor left

What stops the shop from selling the same item twice when two orders arrive at the same moment.

Customer A

Reads monitor: stock 1, version 5

both pass the "enough stock?" check

Saves: stock 0, version 6 WHERE version = 5

1 row updated → 201 Created

Customer B

Reads monitor: stock 1, version 5

Saves: stock 0, version 6 WHERE version = 5

0 rows updated (version is now 6) → 409 "please retry"

Layer 1: stock check

In OrderService . Rejects most late buyers with "available 0".

Layer 2: @Version

On Product . Catches the buyers who arrive at exactly the same instant.

Result of the real test: 20 buyers, 1 monitor

1 × 201 and 19 × 409 . Never oversold.

Your turn

Try it yourself

You need Java 21 and Maven. Claude Code is only needed for step 4.

1. Get the code and start the app

```
git clone https://github.com/tpillai/addysomani-agent-skill-demo.git
cd addysomani-agent-skill-demo
mvn spring-boot:run
```

2. Place an order (second terminal)

```
curl -X POST localhost:8080/api/orders -H 'Content-Type: application/json' -d
'{"lines":[{"productId":1,"quantity":2}]}'
```

3. Run all 17 checks, including the 20-buyer race

```
scripts/smoke-test.sh
```

4. Install the skills in Claude Code

```
claude plugin marketplace add addyosmani/agent-skills
claude plugin install agent-skills@addy-agent-skills
```

Then open Claude Code in the project folder and follow [TRY-IT.md](#) to build the orders feature yourself.

Words you might not know

AI agent

An AI that can edit files and run commands, not just chat.

Skill

A recipe card telling the AI how to do one kind of job well.

Spec

A short design doc: what to build, and what "done" means.

CLAUDE.md

A note for the AI about your project: how to build it and the team's rules. Like onboarding a new teammate.

Mock

A fake object used in tests instead of the real thing, such as a pretend database.

@Transactional

"Do all of this, or none of it." Without it, the stock changes were never saved.

@Version

A counter on each row that stops two people from updating the same item at the same moment.

ProblemDetail

A standard JSON shape for error messages (RFC 7807).

Skills: [addyosmani/agent-skills](#) by Addy Osmani (MIT). Agent: [Claude Code](#) by Anthropic.

Demo code: [tpillai/addysomani-agent-skill-demo](#). A learning project, not production code.